

FrostPuno

Informe Técnico — Aprendizaje de Máquina

Proyecto de la Segunda Unidad

Aplicación con elementos inteligentes basada en aprendizaje NO SUPERVISADO (K-Means), desplegada en producción, con mantenimiento e integración continua automatizados.

Campo	Detalle
Universidad	Universidad Nacional del Altiplano — Ingeniería de Sistemas
Integrantes	[Integrantes]
Docente	[Completar]
Lugar y año	Puno, Perú — 2026

Producto en producción

Componente	URL
Aplicación web (Flutter)	https://frost-puno.vercel.app
API (FastAPI)	https://frost-puno.onrender.com (docs en /docs)
Repositorio	https://github.com/JosephElvisMaman1/frost-puno
APK Android	app-release.apk (47 MB)
Video de la exposición	[completar URL]

1. Resumen

FrostPuno predice el riesgo de heladas en el altiplano de Puno y apoya la producción tradicional de chuño. La lógica de negocio se operativiza mediante un modelo de aprendizaje no supervisado (K-Means) que agrupa los regímenes climáticos de los distritos y los mapea a niveles de riesgo. El sistema está desplegado en producción (Render + Vercel + Supabase) y su mantenimiento —incluido el reentrenamiento del modelo— está automatizado con GitHub Actions, con un quality gate que bloquea modelos que empeoran.

Nota de trazabilidad académica. El proyecto nació con un clasificador supervisado (RandomForest, métrica f1_macro). Se migró a aprendizaje no supervisado conforme al enunciado de la segunda unidad. Los scripts supervisados se conservan como legacy documentado, fuera del path productivo.

2. Problema y contexto

En el altiplano (> 3800 m) las heladas dañan cultivos y causan mortalidad neonatal en crías de alpaca y ovino. Las mismas heladas son el insumo del chuño (papa deshidratada por congelamiento nocturno y secado diurno). Las decisiones se toman por experiencia empírica, sin información climática integrada. FrostPuno entrega: alerta diaria de helada, riesgo diferenciado para ganado, ventanas óptimas de chuño, detección de eventos inusuales e histórico climático.

3. Dataset y características

Fuentes: Open-Meteo (clima horario, histórico y pronóstico), semilla territorial compatible con INEI para los distritos de Puno, y SENAMHI documentado como fuente oficial prioritaria (sin API pública estable en el MVP).

Volumen: 13 distritos × ventana horaria → 4 368 registros en data/processed/frost_training_dataset.csv (versionado en git: actúa como feature store del MVP y se valida en IC).

Características del modelo

Feature	Origen
altitud_estimada	semilla territorial del distrito
temperature_2m	Open-Meteo
dew_point_2m	Open-Meteo
relative_humidity_2m	Open-Meteo

Selección de características (experimento).

Se partió de 8 variables. Un experimento de subconjuntos mostró que precipitation, wind_speed_10m, cloud_cover y apparent_temperature eran ruidosas o colineales y

degradaban la separación de grupos. Al podarlas, la silhouette subió de 0.286 a 0.419 (+47 %). Se conservaron las cuatro variables con sentido físico para helada.

Estas cuatro features están disponibles tanto en entrenamiento como en inferencia en vivo, garantizando que el mismo vector se pueda construir en producción. Este es el contrato ML↔backend, declarado en el metadata del modelo.

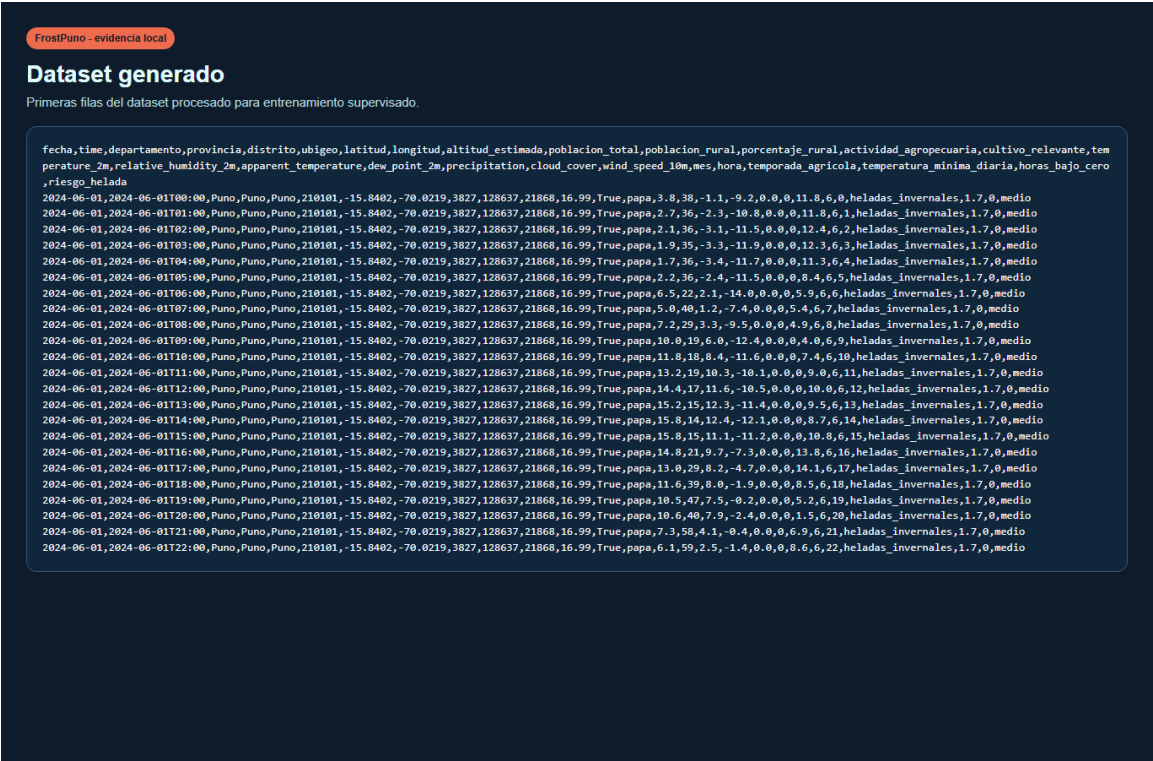


Fig. 1. Dataset de entrenamiento generado.

4. Entrenamiento del modelo e hiperparámetros optimizados

Algoritmo: KMeans (scikit-learn) dentro de un Pipeline con StandardScaler, ya que las features tienen escalas muy distintas (metros vs. grados vs. porcentaje).

Optimización de hiperparámetros: grid search de 24 combinaciones maximizando silhouette.

Hiperparámetro	Valores explorados
n_clusters (k)	3, 4, 5, 6, 7, 8
init	k-means++, random
n_init	10, 25
random_state	42 (reproducibilidad)

Configuración ganadora: k=3, init=random, n_init=10, StandardScaler. El grid completo se persiste en cluster_metadata.json bajo hyperparameter_search como evidencia auditable.

Selección de k (mejor combinación por cada k)

k	Silhouette	Davies-Bouldin
3 (elegido)	0.4195	0.813
4	0.3894	0.906
5	0.3760	0.820
6	0.3697	0.870
7	0.3645	0.865
8	0.3391	0.883

k = 3 gana en silhouette y en Davies-Bouldin, y coincide con los tres niveles de riesgo operativos (alto/medio/bajo).

Asignación de niveles (post-hoc): los clusters se ordenan por temperatura media; el más frío es riesgo alto. Es descripción de los grupos hallados, no supervisión: no se usan etiquetas en el entrenamiento.

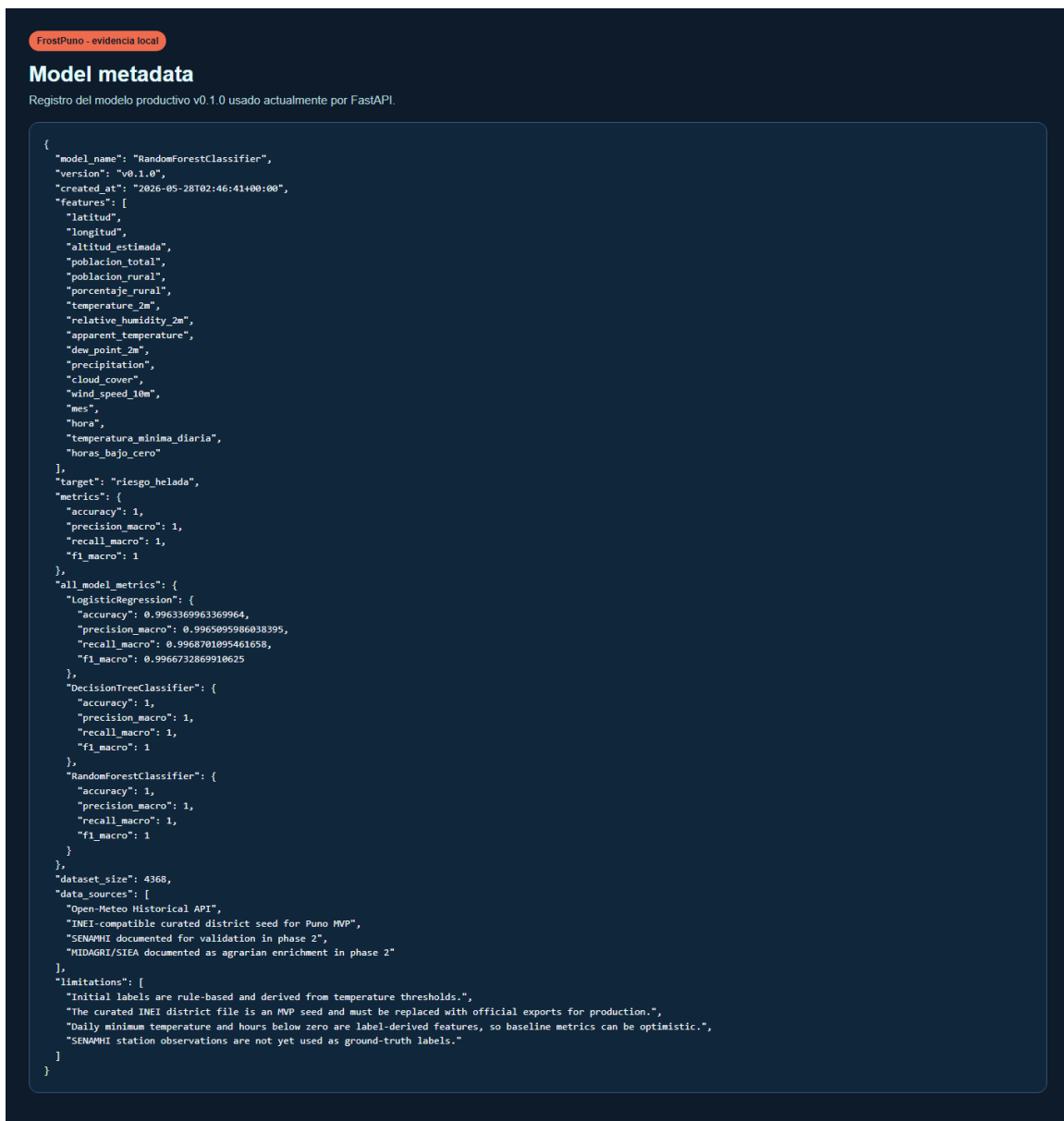


Fig. 2. Metadata del modelo en el registry.

5. Evaluación

Métrica	Valor	Interpretación
Silhouette	0.4195	cohesión/separación de los grupos (mayor es mejor)
Davies-Bouldin	0.813	dispersión intra vs. inter cluster (menor es mejor)
Inercia	5 783	suma de distancias al centroide

Por qué no accuracy ni F1: no existen etiquetas de verdad de terreno para 'hubo helada' en estos distritos. Al ser aprendizaje no supervisado, la evaluación mide la calidad de la estructura descubierta, no el acierto contra una etiqueta.

Perfil de los clusters encontrados

Cluster	Nivel	Temp. media	Altitud media	Observaciones
0	alto	4.0 °C	3 905 m	1 889
2	medio	11.6 °C	3 893 m	2 145
1	bajo	13.6 °C	2 170 m	334

Resultado interpretable: Macusani (4 315 m), Juliaca y Huancané → riesgo alto; Puno, Azángaro y Ayaviri → medio; Sandia (2 170 m, valle) → bajo. El ordenamiento surge del clustering, no de reglas escritas a mano, y coincide con el conocimiento geográfico de la región.

Pretty-print ☐

```
{
  "model_name": "RandomForestClassifier",
  "version": "v0.1.0",
  "created_at": "2026-05-28T02:46:41+00:00",
  "features": [
    "latitud",
    "longitud",
    "altitud_estimada",
    "poblacion_total",
    "poblacion_rural",
    "porcentaje_rural",
    "temperature_2m",
    "relative_humidity_2m",
    "apparent_temperature",
    "dew_point_2m",
    "precipitation",
    "cloud_cover",
    "wind_speed_10m",
    "mes",
    "hora",
    "temperatura_minima_diaria",
    "horas_bajo_cero"
  ],
  "target": "riesgo_helada",
  "metrics": {
    "accuracy": 1.0,
    "precision_macro": 1.0,
    "recall_macro": 1.0,
    "f1_macro": 1.0
  },
  "dataset_size": 4368,
  "data_sources": [
    "Open-Meteo Historical API",
    "INEI-compatible curated district seed for Puno MVP",
    "SENAMHI documented for validation in phase 2",
    "MIDAGRI/SIEA documented as agrarian enrichment in phase 2"
  ],
  "limitations": [
    "Initial labels are rule-based and derived from temperature thresholds.",
    "The curated INEI district file is an MVP seed and must be replaced with official exports for production.",
    "Daily minimum temperature and hours below zero are label-derived features, so baseline metrics can be optimistic.",
    "SENAMHI station observations are not yet used as ground-truth labels."
  ]
}
```

Fig. 3. Endpoint /ml/model-info en producción.

6. Organización del código fuente

```
ml_pipeline/      # Pipeline de ML
  config.py       # fuente unica de rutas, features y constantes
  data_ingestion/ # ingesta paralela Open-Meteo (ThreadPoolExecutor)
  features/       # construccion del dataset (feature store)
  clustering/     # train_clusters.py (K-Means) + adaptive_retrain.py
  registry/      # modelo .joblib + metadata + quality gate
```

```
tests/          # pruebas de mantenimiento e IC
backend_fastapi/ # API que sirve el modelo
  app/api/routes/ # HTTP -> services -> repositories -> schemas
app_flutter/    # Cliente web + Android (feature-first, i18n ES/EN)
data/          # datasets versionados y validados en IC
.github/workflows/ # 5 pipelines de integracion continua
```

Principios: configuración centralizada (nada de rutas hardcodeadas), separación estricta por capas en el backend, y el metadata del modelo como contrato entre el pipeline de ML y el servicio de inferencia.

Estructura del proyecto

Arbol generado localmente con carpetas principales y archivos relevantes.

FrostPuno - estructura del proyecto
 Generado localmente para evidencias del informe.
 Raiz: frost-puno

```
|-- .github
|   |-- workflows
|   |   |-- backend-tests.yml
|   |   |-- data-validation.yml
|   |   |-- flutter-build.yml
|   |   |-- ml-training.yml
|   |   |-- model-quality-gate.yml
|-- app_flutter
|   |-- android
|   |   |-- app
|   |   |   |-- src
|   |   |   |   |-- debug
|   |   |   |   |-- main
|   |   |   |   |-- profile
|   |   |   |-- build.gradle.kts
|   |   |-- gradle
|   |   |   |-- wrapper
|   |   |   |   |-- gradle-wrapper.jar
|   |   |   |   |-- gradle-wrapper.properties
|   |   |-- .gitignore
|   |   |-- app_flutter_android.iml
|   |   |-- build.gradle.kts
|   |   |-- gradle.properties
|   |   |-- gradlew
|   |   |-- gradlew.bat
|   |   |-- local.properties
|   |   |-- settings.gradle.kts
|-- lib
|   |-- core
|   |   |-- api
|   |   |   |-- api_client.dart
|   |   |   |-- frost_api_service.dart
|   |   |-- config
|   |   |   |-- app_config.dart
|   |   |-- models
|   |   |   |-- current_weather.dart
|   |   |   |-- health_status.dart
|   |   |-- theme
|   |   |   |-- app_colors.dart
|   |   |   |-- app_theme.dart
|   |   |   |-- theme_mode_scope.dart
|   |-- features
|   |   |-- data_sources
|   |   |   |-- models
|   |   |   |-- screens
|   |   |-- history
|   |   |   |-- models
|   |   |   |-- screens
|   |   |-- home
|   |   |   |-- screens
|   |   |-- location
|   |   |   |-- models
|   |   |   |-- services
|   |   |-- model_info
|   |   |   |-- models
|   |   |   |-- screens
|   |   |-- prediction
|   |   |   |-- models
|   |   |   |-- screens
|   |   |-- shell
|   |   |   |-- app_shell.dart
|   |-- shared
|   |   |-- widgets
|   |   |   |-- frost_app_bar.dart
|   |   |   |-- glass_card.dart
|   |   |   |-- page_scaffold.dart
|   |   |   |-- primary_action_button.dart
|   |   |   |-- responsive_content.dart
|   |   |   |-- section_header.dart
|   |   |   |-- status_chip.dart
|   |-- app.dart
|   |-- main.dart
|-- test
|   |-- widget_test.dart
|-- web
|   |-- icons
|   |   |-- Icon-192.png
|   |   |-- Icon-512.png
|   |   |-- Icon-maskable-192.png
|   |   |-- Icon-maskable-512.png
|   |-- favicon.png
|   |-- index.html
|   |-- manifest.json
|   |-- offline.html
|-- .flutter-plugins-dependencies
|-- .gitignore
|-- .metadata
|-- analysis_options.yaml
|-- app_flutter.iml
|-- pubspec.lock
|-- pubspec.yaml
|-- README.md
|-- static_web.err
|-- static_web.log
|-- vercel.json
-- backend_fastapi
|   |-- app
|   |   |-- api
|   |   |   |-- routes
|   |   |   |   |-- __init__.py
|   |   |   |   |-- health.py
|   |   |   |   |-- ml.py
|   |   |   |   |-- predict.py
|   |   |   |   |-- predictions.py
|   |   |   |   |-- weather.py
|   |   |   |-- __init__.py
|   |   |-- core
|   |   |   |-- __init__.py
```

Fig. 4. Estructura del proyecto.

7. Herramientas y plataformas

Área	Herramienta
ML	Python 3.11, scikit-learn, pandas, joblib
Backend	FastAPI, Uvicorn, Pydantic, httpx
Cliente	Flutter (Web + Android), fl_chart, shared_preferences
Despliegue	Render (API), Vercel (web), Supabase (PostgreSQL)
CI/CD	GitHub Actions (5 workflows)
Datos	Open-Meteo API (histórico, pronóstico y archivo)

8. Consideraciones de despliegue inicial

Backend (Render).

Definido como infraestructura como código en render.yaml: runtime Python 3.11, buildCommand con requirements.txt, healthCheckPath /health y autoDeploy activo. Variables de entorno: MODEL_PATH, MODEL_METADATA_PATH, DISTRICT_CLUSTERS_PATH, CORS_ORIGINS, ENABLE_SUPABASE y SUPABASE_SERVICE_ROLE_KEY (marcada sync:false, nunca en el repositorio).

Lección aprendida en producción: las variables definidas manualmente en el panel de Render prevalecen sobre render.yaml. Tras migrar a K-Means, la API siguió sirviendo el modelo anterior hasta corregir esas variables en el panel. Punto crítico para el equipo de TI que reciba el sistema.

Cliente web (Vercel): app_flutter/vercel.json; API_BASE_URL se inyecta con --dart-define, nunca hardcodeado. Auto-deploy desde main.

APK Android:

```
flutter build apk --release --dart-define=API_BASE_URL=https://frost-puno.onrender.com
```

Persistencia: Supabase es opcional; con ENABLE_SUPABASE=false se usa un repositorio en memoria (así corren los tests en IC).

Orden de puesta en marcha: entrenar y publicar el modelo → desplegar API y verificar /health y /ml/model-info → desplegar cliente con la URL de la API → (opcional) activar Supabase.

Despliegue Render + Vercel + Supabase

Guía de despliegue gratuito con backend FastAPI, Flutter Web, Supabase y GitHub Actions.

FrostPuno - guía de despliegue gratuito

Esta guía prepara el despliegue manual end-to-end del MVP:

- Backend FastAPI en Render Free.
- Flutter Web en Vercel Hobby.
- Base de datos en Supabase Free.
- CI/CD con GitHub Actions.

No se incluyen credenciales reales. El despliegue productivo debe configurarse desde GitHub, Render, Vercel y Supabase.

1. Supabase Free

Crear proyecto

1. Entra al dashboard de Supabase.
2. Crea un proyecto nuevo, por ejemplo 'frost-puno'.
3. Espera a que la base PostgreSQL quede lista.
4. Abre 'SQL Editor'.

Ejecutar SQL

Ejecuta en este orden:

1. 'supabase/migrations/001_create_core_tables.sql'
2. 'supabase/policies/pls_policies.sql'
3. 'supabase/seed/seed_locations.sql'

Copiar credenciales

En 'Project Settings > API' copia:

- 'Project URL': se usara como 'SUPABASE_URL'.
- 'service_role key': se usara solo en Render como 'SUPABASE_SERVICE_ROLE_KEY'.

Nunca colocar 'SUPABASE_SERVICE_ROLE_KEY' en Flutter, Vercel, 'dart-define', JavaScript publico ni archivos versionados.

2. Render Free - FastAPI

Estado actual verificado

- Backend productivo: 'https://frost-puno.onrender.com'
- Endpoint de salud: 'https://frost-puno.onrender.com/health'
- Documentacion Swagger: 'https://frost-puno.onrender.com/docs'
- Modelo activo: 'v0.1.0'
- Rama productiva usada por Render: 'main'

La rama 'codex/model-v0-2-0' contiene la evaluacion academica 'v0.2.0', pero no reemplaza el modelo activo del backend productivo.

Opcion A: Blueprint v0.1.0 productivo

El repositorio incluye 'render.yaml'. En Render:

1. Crea un nuevo Blueprint desde el repositorio de GitHub.
2. Render detectara 'render.yaml'.
3. Revisa el servicio 'frost-puno-api'.
4. Configura las variables faltantes.
5. Despliega.

Opcion B: Blueprint v0.2.0 experimental

El repositorio incluye 'render.v2.yaml' para levantar un servicio separado:

- **Nombre sugerido:** 'frost-puno-api-v2'
- **Rama sugerida:** 'codex/model-v0-2-0'
- **Modelo:** 'ml_pipeline/registry/frost_risk_model_v0_2_0.joblib'
- **Metadata:** 'ml_pipeline/registry/model_metadata_v0_2_0.json'
- **AutoDeploy:** desactivado para evitar promover cambios experimentales sin revision.

Esta opcion permite exponer 'v0.2.0' como endpoint academico sin romper el backend actual. La URL esperada, una vez creado el servicio en Render, seria similar a:

```
'''text
https://frost-puno-api-v2.onrender.com
'''
```

Opcion C: Web Service manual

Configuracion recomendada:

- **Environment:** Python
- **Root Directory:** raiz del repositorio
- **Build Command:**

```
'''bash
pip install --upgrade pip && pip install -r backend_fastapi/requirements.txt
'''
```

- **Start Command:**

```
'''bash
uvicorn app.main:app --host 0.0.0.0 --port $PORT --app-dir backend_fastapi
'''
```

- **Health Check Path:** '/health'

Variables de entorno Render

```
'''env
PYTHON_VERSION=3.11.9
ENABLE_SUPABASE=false
SUPABASE_URL=https://TU-PROYECTO.supabase.co
SUPABASE_SERVICE_ROLE_KEY=TU_SERVICE_ROLE_SOLO_BACKEND
CORS_ORIGINS=https://TU-FRONTEND.vercel.app,http://localhost:3000,http://127.0.0.1:5174,http://localhost:5174
MODEL_PATH=ml_pipeline/registry/frost_risk_model_v0_2_0.joblib
MODEL_METADATA_PATH=ml_pipeline/registry/model_metadata.json
'''
```

Para el servicio experimental 'v0.2.0', cambiar solo:

```
'''env
MODEL_PATH=ml_pipeline/registry/frost_risk_model_v0_2_0.joblib
MODEL_METADATA_PATH=ml_pipeline/registry/model_metadata_v0_2_0.json
```

Fig. 5. Despliegue en Render, Vercel y Supabase.

Android APK y prueba movil

Guia para correr FrostPuno en Android, configurar SDK local, probar GPS y generar APK.

```
# FrostPuno - build movil y PWA

Esta guia prepara FrostPuno para Android APK, App Bundle y PWA sin introducir claves privadas ni firmar releases definitivos.

## Requisitos locales

- Flutter estable instalado.
- Android SDK configurado.
- Un dispositivo Android o emulador.
- Backend local o desplegado en Render.

Verificacion:

```powershell
flutter doctor -v
```

Si aparece 'Unable to locate Android SDK', instalar Android Studio o configurar:

```powershell
flutter config --android-sdk "C:\Ruta\Al\Android\Sdk"
$env:ANDROID_HOME = "C:\Ruta\Al\Android\Sdk"
```

## Permisos Android

El 'AndroidManifest.xml' declara 'INTERNET', 'ACCESS_COARSE_LOCATION' y 'ACCESS_FINE_LOCATION'.

La app solicita permiso de ubicacion en tiempo de ejecucion mediante 'permission_handler' y obtiene coordenadas con 'geolocator'.

## Icono APK

El launcher icon ya fue reemplazado por un icono propio de FrostPuno en:

```text
app_flutter/android/app/src/main/res/mipmap-*/ic_launcher.png
```

La PWA usa la misma identidad visual en:

```text
app_flutter/web/icons/
app_flutter/web/favicon.png
```

## Ejecutar en Android

```powershell
Set-Location app_flutter
flutter run -d emulator --dart-define=API_BASE_URL=http://10.0.2.2:8000
```

Para dispositivo fisico con backend desplegado:

```powershell
flutter run -d android --dart-define=API_BASE_URL=https://frost-puno.onrender.com
```

## Generar APK

```powershell
Set-Location app_flutter
flutter build apk --release --dart-define=API_BASE_URL=https://frost-puno.onrender.com
```

Salida esperada:

```text
app_flutter/build/app/outputs/Flutter-apk/app-release.apk
```

## Generar App Bundle

```powershell
flutter build appbundle --release --dart-define=API_BASE_URL=https://frost-puno.onrender.com
```

## Probar GPS

1. Abrir FrostPuno en Android o Flutter Web HTTPS.
2. Ir a 'Consultar riesgo'.
3. Tocar 'Usar mi ubicacion'.
4. Aceptar permiso de ubicacion.
5. Confirmar que latitud/longitud cambien.
6. Confirmar que el bloque de clima indique proveedor y fallback.
7. Ejecutar prediccion.

## PWA

El proyecto incluye 'web/manifest.json', 'web/offline.html' y service worker generado por Flutter durante 'flutter build web'.

```powershell
flutter build web --release --dart-define=API_BASE_URL=https://frost-puno.onrender.com
```

## Limitaciones actuales

- No hay modo offline-first complejo; clima y predicciones requieren red.
- El APK no esta firmado para Play Store.
- GPS en navegador requiere HTTPS y permiso del usuario.
- SENAMHI esta preparado como proveedor prioritario, pero el MVP usa Open-Meteo como fallback cuando no hay API publica estable configurada.
```

Fig. 6. Aplicación Android desde el APK release.

9. Flujos de mantenimiento e integración continua

| Workflow | Función |
|------------------------|--|
| ml-training.yml | Ingesta → features → entrenamiento → artefactos. Cron semanal. |
| model-quality-gate.yml | Barrera: bloquea silhouette < 0.35 + pruebas de mantenimiento |
| backend-tests.yml | 12 pruebas de la API sin Supabase |
| data-validation.yml | Valida datasets y agrupación de distritos |
| flutter-build.yml | analyze + test + build web |

Mantenimiento del elemento inteligente:

- Reentrenamiento automatizado: ml-training.yml corre por schedule (cron semanal) sin intervención humana.
- Reentrenamiento adaptativo — el modelo sigue aprendiendo: adaptive_retrain.py reentrena con datos frescos y, si la silhouette no alcanza el objetivo (0.45), amplía la ventana de datos (14 → 29 → 44 días) y reintenta. Cada corrida se registra en registry/training_history.json.
- Registro de modelos (registry): cada entrenamiento versiona modelo + metadata (features, métricas, hiperparámetros, limitaciones).
- Almacenamiento de características: el dataset de features está versionado en data/processed/ y se valida en IC.
- Quality gate: ningún modelo se promueve si silhouette < 0.35.

Platform Solutions Resources Open Source Enterprise Pricing

Search or jump to.../Sign inSign up

JosephElvisMaman1 / frost-punoPublic

NotificationsFork 0Star 0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

Actions

All workflows

Backend Tests

Data Validation

Flutter Build

ML Training

Model Quality Gate

Management

Caches

Deployments

All workflows

Showing runs from all workflows

Filter workflow runs

| 20 workflow runs | Workflow | Event | Status | Branch | Actor |
|--|----------|-------|--------|--------|-------|
| <div>docs: completar informe y evidencias de despliegue</div> <div>Backend Tests #5: Commit e1e03bd pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 8:15 AM</div> <div>32s</div> <div>...</div> | | | | | |
| <div>docs: completar informe y evidencias de despliegue</div> <div>Model Quality Gate #5: Commit e1e03bd pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 8:15 AM</div> <div>8s</div> <div>...</div> | | | | | |
| <div>docs: completar informe y evidencias de despliegue</div> <div>Flutter Build #5: Commit e1e03bd pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 8:15 AM</div> <div>1m 23s</div> <div>...</div> | | | | | |
| <div>docs: completar informe y evidencias de despliegue</div> <div>Data Validation #5: Commit e1e03bd pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 8:15 AM</div> <div>27s</div> <div>...</div> | | | | | |
| <div>feat: agregar modelo experimental v0.2.0</div> <div>Flutter Build #4: Commit f336179 pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 2:28 AM</div> <div>1m 27s</div> <div>...</div> | | | | | |
| <div>feat: agregar modelo experimental v0.2.0</div> <div>Model Quality Gate #4: Commit f336179 pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 2:28 AM</div> <div>11s</div> <div>...</div> | | | | | |
| <div>feat: agregar modelo experimental v0.2.0</div> <div>Data Validation #4: Commit f336179 pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 2:28 AM</div> <div>29s</div> <div>...</div> | | | | | |
| <div>feat: agregar modelo experimental v0.2.0</div> <div>Backend Tests #4: Commit f336179 pushed by JosephElvisMaman1</div> <div>codex/model1-v0-2-0</div> <div>Today at 2:28 AM</div> <div>31s</div> <div>...</div> | | | | | |
| <div>fix: configurar build Android local</div> <div>Model Quality Gate #3: Commit 87bd81f pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 1:52 AM</div> <div>11s</div> <div>...</div> | | | | | |
| <div>fix: configurar build Android local</div> <div>Backend Tests #3: Commit 87bd81f pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 1:52 AM</div> <div>30s</div> <div>...</div> | | | | | |
| <div>fix: configurar build Android local</div> <div>Data Validation #3: Commit 87bd81f pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 1:52 AM</div> <div>29s</div> <div>...</div> | | | | | |
| <div>fix: configurar build Android local</div> <div>Flutter Build #3: Commit 87bd81f pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 1:52 AM</div> <div>1m 19s</div> <div>...</div> | | | | | |
| <div>feat: mejorar experiencia movil FrostPuno</div> <div>Data Validation #2: Commit 0971225 pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 12:56 AM</div> <div>30s</div> <div>...</div> | | | | | |
| <div>feat: mejorar experiencia movil FrostPuno</div> <div>Flutter Build #2: Commit 0971225 pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 12:56 AM</div> <div>1m 27s</div> <div>...</div> | | | | | |
| <div>feat: mejorar experiencia movil FrostPuno</div> <div>Model Quality Gate #2: Commit 0971225 pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 12:56 AM</div> <div>8s</div> <div>...</div> | | | | | |
| <div>feat: mejorar experiencia movil FrostPuno</div> <div>Backend Tests #2: Commit 0971225 pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 12:56 AM</div> <div>34s</div> <div>...</div> | | | | | |
| <div>Prepare Frost Puno MVP for deployment</div> <div>Backend Tests #1: Commit e4df8bf pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 7:41 PM</div> <div>39s</div> <div>...</div> | | | | | |
| <div>Prepare Frost Puno MVP for deployment</div> <div>Data Validation #1: Commit e4df8bf pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 7:41 PM</div> <div>37s</div> <div>...</div> | | | | | |
| <div>Prepare Frost Puno MVP for deployment</div> <div>Model Quality Gate #1: Commit e4df8bf pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 7:41 PM</div> <div>8s</div> <div>...</div> | | | | | |
| <div>Prepare Frost Puno MVP for deployment</div> <div>Flutter Build #1: Commit e4df8bf pushed by JosephElvisMaman1</div> <div>main</div> <div>Today at 7:41 PM</div> <div>2m 35s</div> <div>...</div> | | | | | |

Fig. 7. Workflows de integración continua en GitHub Actions.

10. Pruebas de funcionamiento del mantenimiento e IC

ml_pipeline/tests/test_maintenance_ci.py — 6 pruebas que corren en IC:

- El reentrenamiento deja modelo, metadata y agrupación de distritos.
- El .joblib se carga e infiere (contrato con el backend intacto).
- Los 13 distritos quedan asignados a un tier válido.
- La metadata documenta el grid search y la configuración elegida es la mejor del grid.
- El quality gate aprueba el modelo en producción.
- El quality gate BLOQUEA un modelo degradado (silhouette 0.10 → exit code 1): la integración se detiene ante una regresión del modelo.

Demostración en vivo del gate:

```
python -m ml_pipeline.registry.check_cluster_quality --min-silhouette 0.35  #  
PASSED  
python -m ml_pipeline.registry.check_cluster_quality --min-silhouette 0.99  #  
FAILED, exit 1
```

Otras evidencias: 12 pruebas del backend en verde, flutter analyze/test/build web en verde, APK release compilando, y el historial de corridas en la pestaña Actions. Detalle completo en docs/pruebas_mantenimiento_ic.md.

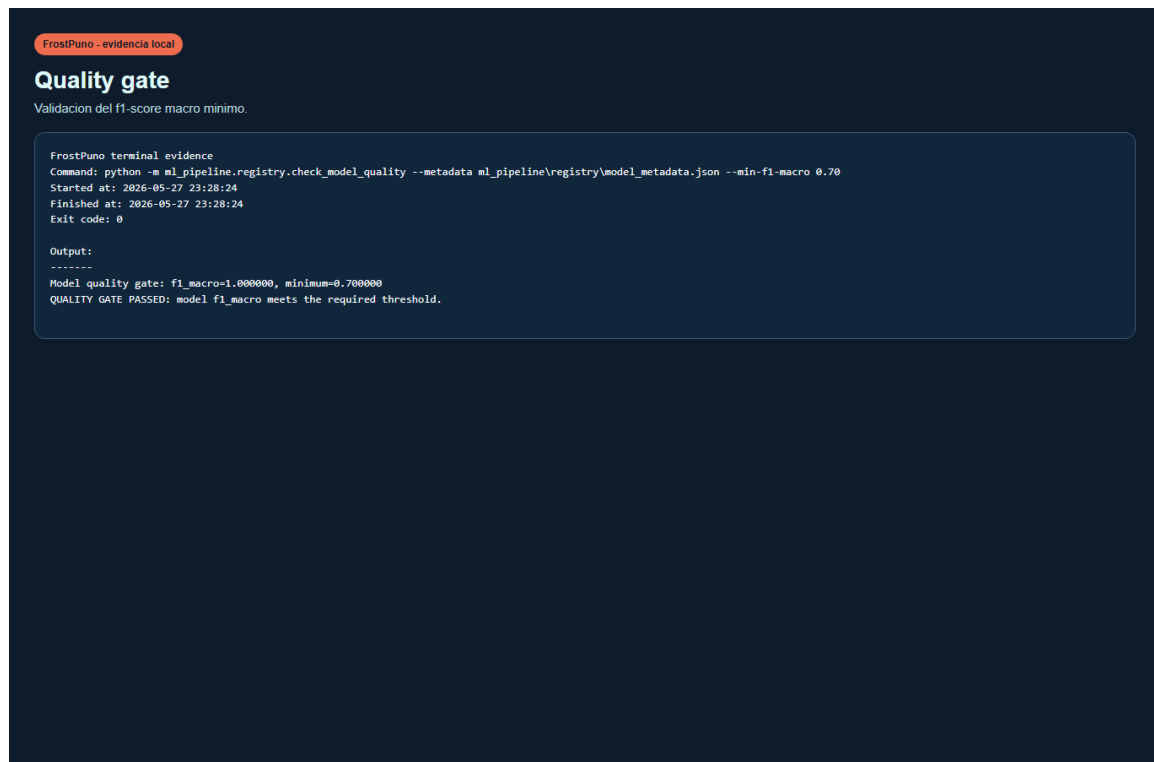


Fig. 8. Ejecución del quality gate.

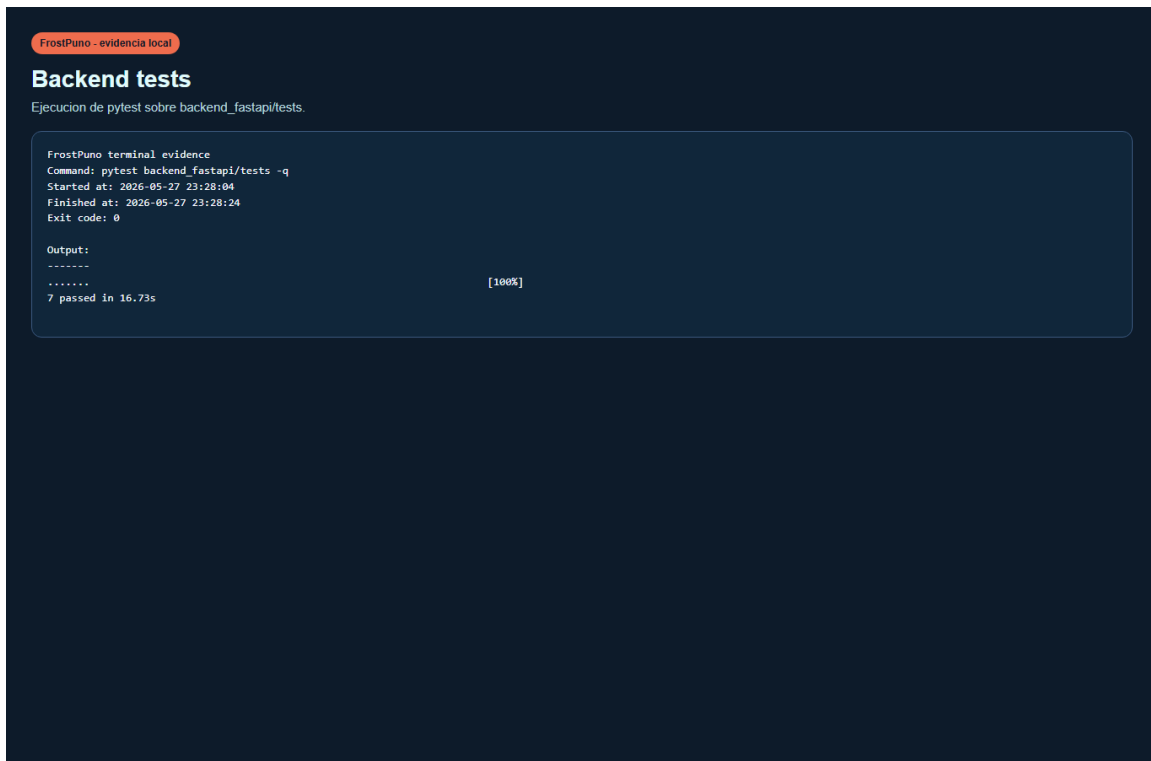


Fig. 9. Pruebas del backend en verde.

11. Funcionamiento de la aplicación (demo en inglés)

La aplicación incluye modo inglés (Ajustes → Language → English): navegación, alertas, niveles de riesgo y recomendaciones se muestran en inglés. El guión palabra por palabra para la exposición está en docs/english_demo_script.md.

Funcionalidades: alerta diaria de helada con umbral configurable, tarjeta de riesgo para ganado, detección de anomalías, zonas de riesgo por K-Means, módulo estacional de chuño, histórico climático y perfiles de usuario.

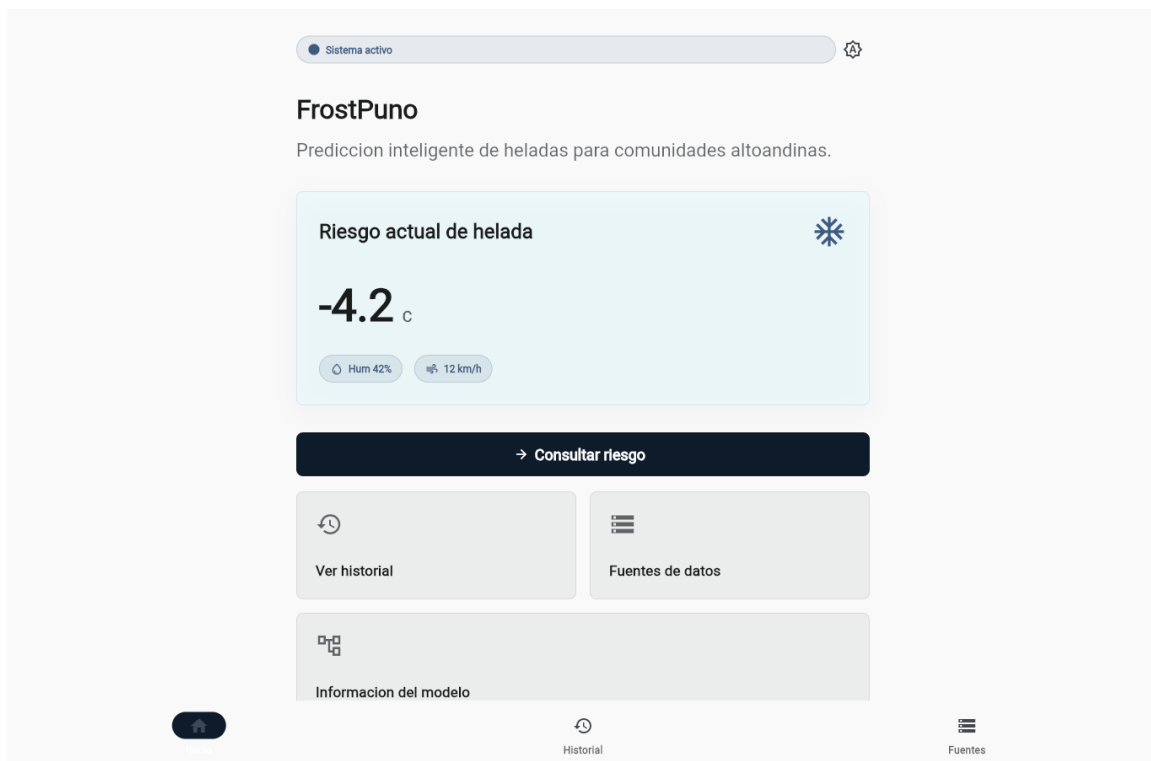


Fig. 10. Aplicación desplegada en Vercel.

12. Limitaciones

- Los niveles de riesgo derivan del perfil térmico de cada cluster, no de etiquetas oficiales de heladas observadas.
- SENAMHI no expone API pública estable; Open-Meteo es la fuente operativa real.
- La semilla de distritos es un MVP y debe reemplazarse por exportes oficiales del INEI.
- Los umbrales de chuño y ganado son diseño MVP documentado (docs/chuno_criteria.md, docs/livestock_criteria.md).
- La detección de anomalías usa climatología reciente (30 días), no serie multianual.
- Render Free tiene arranque en frío (primera petición lenta).

13. Conclusiones

FrostPuno cumple el objetivo de la unidad: una aplicación con elementos inteligentes basados en aprendizaje no supervisado, desplegada en producción y con procesos de mantenimiento e integración continua automatizados y probados. El modelo K-Means alcanza silhouette 0.419 tras selección de características y optimización de hiperparámetros; el sistema se reentrena solo, registra su historial de calidad y se protege con un quality gate verificado por pruebas.